

EpstLoop: An Epistemic Transformer Belief Loop

Implementing and validating a source-weighted, time-decaying belief store that gates transformer attention and makes internal reasoning cost proportional to epistemic uncertainty.

Author: Chiru Dhung **Date:** September 15, 2026 **Period covered:** Aug 23 – Sep 2, 2026

Status: Core mechanisms complete & tested

1 Project Overview

EpstLoop is a working implementation of the four mechanisms that make up the **Epistemic Transformer (EpT)** architecture — a modification to the standard transformer that replaces implicit, weight-baked factual knowledge with an **explicit, auditable, revisable belief store**.

In a conventional LLM, a fact and a guess are stored the same way: as weights. There is no record of where a claim came from, no mechanism for it to go stale, and no way to revise it short of retraining. EpT attacks that by holding beliefs in a separate structure where every entry carries provenance, a confidence weight, and a decay rate — then using that confidence to modulate the forward pass itself.

The project's **main goal** is a functioning epistemic loop: beliefs are retrieved and weighted during a forward pass, low confidence triggers extra internal reasoning, and the model's own output feeds back to strengthen, contest, or suppress the beliefs that produced it. Four components, corresponding to Sections 3.1–3.5 of the design specification:

The four EpT mechanisms and what each contributes

§	MECHANISM	FUNCTION IN THE LOOP
3.1	Belief Store	Holds belief entries addressed by sparse autoencoder (SAE) concept vectors. Each has a weight, a source tier, a content type governing its decay rate, and a lifecycle state.
3.3	Belief Gate	Extracts concepts from attention output, looks up their effective belief weight, and scales attention output by that confidence: $z_{\text{gated}} = z_{\text{attn}} \odot w_{\text{eff}}$.
3.4	FOK Gate	Feeling-of-Knowing. Computes a confidence signal from belief weights and runs expensive internal reasoning <i>only</i> on tokens below a threshold.
3.5	Reconsolidation	Compares generated claims against retrieved beliefs via natural language inference; reinforces, contests, or supersedes entries, and applies retrieval-induced forgetting to competitors.

The central design claim being tested is that **reasoning overhead should scale with uncertainty, not with token count**. Quiet-STaR-style approaches pay rationale-generation cost uniformly on every token; EpT only pays it where belief confidence is low.

2 Work Completed

4 / 4	1,290	17	537x
-------	-------	----	------

Core mechanisms implemented

Lines of Python, 10 modules

Tests passing (44 assertions)

Retrieval throughput gain

All four mechanisms are implemented, individually tested, and verified to compose end-to-end on Apple Silicon GPU (MPS). Three correctness defects in the original specification were found and fixed in the process.

Module inventory (verified via `wc -l`)

MODULE	LINES	CONTENTS
belief_store.py	311	§3.1 — <code>BeliefEntry</code> , <code>BeliefStore</code> , trust/decay tables, <code>query</code> , <code>query_batch</code> , <code>k_nearest</code> , <code>apply_rif</code>
reconsolidation.py	202	§3.5 — <code>run_reconsolidation_pass</code> , <code>MockNLI</code> , <code>HFNLIClassifier</code>
test_belief_store.py	150	6 tests: state semantics, RIF floor, batch/loop equivalence
test_reconsolidation.py	150	6 tests: NLI branches, skip paths, RIF firing
belief_gate.py	122	§3.3 — <code>BeliefGate</code> , <code>TopKSAEEncoder</code> (frozen)
fok_gate.py	119	§3.4 — <code>FOKGate</code> with gather/scatter conditional compute
smoke_test_fok_gate.py	111	4 tests incl. autograd and compute-saving verification
bench_query.py	48	Retrieval throughput benchmark harness
smoke_test_belief_gate.py	42	Shape / device / flagging smoke test
check_device.py	35	MPS / CUDA / CPU device resolution
Total	1,290	

2.1 Belief Store (§3.1)

The store is a dictionary of `BeliefEntry` records keyed by ID, each addressed by an SAE feature vector. Retrieval is cosine nearest-neighbour over concept keys. The heart of it is the effective-weight function, which combines three independent signals — raw belief strength, how much the source is trusted, and how stale the entry is:

```
def effective_weight(self, t_now: int) -> float:
    """w_eff(e, t) with Fix 1 applied for state handling."""
    if self.state == BeliefState.SUPERSEDED:
        return 0.0 # hard exclude – successor entry should be retrieved instead

    base = (
        self.weight
        * SOURCE_TRUST[self.source_tier]
        * math.exp(-CONTENT_DECAY[self.content_type] * (t_now - self.last_accessed))
    )

    if self.state == BeliefState.CONTESTED:
        return base * CONTESTED_PENALTY # reduced, not zero
    return base # ACTIVE
```

Source trust is tiered — `OBSERVED` 1.00, `VERIFIED` 0.90, `STATED` 0.85, `INFERRED` 0.70 — so a belief the system inferred for itself is structurally less trusted than one it directly observed. Decay is content-dependent, with half-lives chosen to match how fast different kinds of knowledge actually go stale:

ARCHITECTURE 730 days, IMPLEMENTATION 120 days, STATUS 14 days. A claim about project status decays roughly 52× faster than a claim about architecture.

2.2 Belief Gate (§3.3)

Implements $c = \text{TopK}(W_{\text{enc}}(z_{\text{attn}} - b_{\text{pre}}) + b_{\text{enc}})$ as a frozen top- k SAE encoder, then gates attention output by the retrieved weight. A deliberate architectural decision here: the belief store stays on **CPU** while only tensor math runs on GPU. Dictionary lookups and small nearest-neighbour searches get no useful GPU speedup, so forcing the store onto the device would add transfer overhead for nothing. The gate does one off-device transfer per forward pass for the whole block, not one per token.

2.3 FOK Gate (§3.4)

This is where the compute-proportional-to-uncertainty claim either holds or doesn't. The naive implementation — compute rationales for every token, then mask the confident ones away — would satisfy the equations while completely defeating the purpose. Instead the gate *gathers* only triggered rows, runs the reasoning module on that subset, and scatters results back via out-of-place `index_copy + torch.where` to keep the autograd graph intact:

```
idx = mask_flat.nonzero(as_tuple=True)[0]
if idx.numel() == 0:
    # nothing triggered this pass – skip thought_fn/mixing_head
    # entirely rather than running them on zero rows
    return zffn, PIK, trigger_mask

selected = zffn_flat.index_select(0, idx) # (T, D) – only the uncertain tokens
thought = self.thought_fn(selected)      # (T, D)
combined = selected + thought
mixed = self.mixing_head(torch.cat([selected, combined], dim=-1))
```

A dedicated test instruments `thought_fn` with a call counter and asserts it is invoked **zero** times on an all-confident batch — so the saving is verified behaviourally, not just asserted in a comment.

2.4 Selective Reconsolidation (§3.5)

`run_reconsolidation_pass` consumes the entries flagged during the forward pass and applies the epistemic update: **CONTRADICTION** marks the old entry **CONTESTED** and creates an **INFERRED** successor at weight 0.5 with a `supersedes` provenance link; **ENTAILMENT** reinforces the existing weight, capped at 1.0. Retrieval-induced forgetting fires on **every** branch, since retrieval happened regardless of the comparison outcome. Two NLI backends are provided: `MockNLI` (deterministic word-overlap heuristic, for plumbing tests) and `HFNLIClassifier` (DeBERTa-v3 MNLI, for real use).

The function returns a five-way summary — `contested`, `reinforced`, `neutral`, `skipped_no_claim`, `skipped_no_text` — so entries that could not be processed are reported explicitly rather than silently dropped.

2.5 Three specification defects found and fixed

Implementing the spec surfaced three genuine correctness bugs. These are the most substantive results of the period.

FIX 1 Effective-weight state handling

Spec as written

`w_eff` includes a factor `1[state == ACTIVE]`, which sends `CONTESTED` entries to exactly zero — identical treatment to `SUPERSEDED`.

Why that is wrong

A contested belief is precisely the case where the model most needs to retrieve it, at reduced confidence, so downstream components can reason about the conflict. Zeroing it makes the contradiction invisible. Conversely `SUPERSEDED` should be excluded *entirely*, since a successor entry exists that ought to be retrieved instead.

Fix

`SUPERSEDED` → hard exclude (0.0). `CONTESTED` → multiply by a configurable penalty (default 0.5). `ACTIVE` → full weight.

FIX 2 Unbounded retrieval-induced forgetting

Spec as written

`entry.weight -= eta_inhibit * cos(c, n.concept_key)`, with no lower bound.

Why that is wrong

Repeated retrieval of semantically related concepts drives the weight **negative**, and that negative value propagates a **sign flip** into `z_gated = z_attn ⊗ w_eff`. RIF would not merely suppress a competing belief — it would *invert* it. This is a silent correctness failure, not an edge case.

Fix

Clamp to `[RIF_FLOOR, 1.0]` with `RIF_FLOOR = 0.02` — deliberately a small positive value, not zero, so "heavily suppressed" stays distinguishable from "does not exist." Full removal remains an explicit act of supersession, never an emergent side effect of gradual inhibition.

FIX 3 Intra-pass decay reset

Bug

The per-token `query()` mutated `entry.last_accessed = t_now` as a retrieval side effect. If two tokens in the *same* forward pass hit the same entry, the second computed decay as `exp(-λ · 0)` — no decay at all — and reported the stale belief as perfectly fresh.

Fix

`query_batch()` computes all weights from a pre-pass snapshot and applies bookkeeping only afterward, so every token in a pass sees consistent state. A regression test pins the correct behaviour and reproduces the old bug numerically for comparison.

2.6 Retrieval throughput

The original per-token design meant a separate Python-level dictionary traversal and cosine search for every token — at sequence length 2048, over 2,000 independent `store.query()` calls per forward pass. `query_batch()` replaces this with a single $(M,D) \times (D,N)$ matrix multiply. Same asymptotic work, but vectorised in NumPy instead of paying interpreter overhead N times.

3 Evidence of Progress

All results below were produced on the dates shown by running the project's own test and benchmark scripts. Environment: Python 3.12.6, PyTorch 2.13.0, NumPy 2.5.2, device **MPS** (Apple Silicon GPU).

3.1 Full test suite — 17 checks, all passing

```
$ python test_belief_store.py
[PASS] CONTESTED weff=0.3600 (penalized from 0.7200, not zeroed)
[PASS] SUPERSEDED weff is hard 0.0, as intended
[PASS] After 50 rounds of RIF, neighbor.weight=0.0200 (floored at 0.02, never negative)
[INFO] Old unbounded spec after 50 rounds: weight=-4.2500 (negative -> would flip zgated sign)
[PASS] query_batch matches query() when no entry is hit twice in-pass (max diff=7.52e-08)
[PASS] Old loop bug reproduced: weff[0]=0.5309 -> weff[1]=0.6800 (2nd query wrongly 'fresher').
        Batched: weff[0]=0.5309 == weff[1]=0.5309
```

All tests passed.

```
$ python test_reconsolidation.py
[PASS] contradiction -> old entry CONTESTED (weff=0.3498, not 0), new INFERRED entry created
        with supersedes link
[PASS] entailment -> weight reinforced 0.6 -> 0.7000
[PASS] reinforcement capped at 1.0 (got 1.0000)
[PASS] flagged entry with no supplied claim is skipped and reported, not silently ignored
[PASS] missing content_text -> skipped for NLI but RIF still applied (neighbor 0.5 -> 0.4402)
[PASS] RIF fired on contradiction branch (neighbor weight 0.5 -> 0.3912)
[PASS] RIF fired on entailment branch (neighbor weight 0.5 -> 0.4121)
```

All reconsolidation tests passed.

```
$ python smoke_test_fok_gate.py
Using device: mps
[PASS] basic shapes/trigger: 8/16 tokens triggered, untouched tokens pass through exactly,
        triggered tokens modified
[PASS] zero triggers: thought_fn never called, output identical to input
[PASS] all triggered: shapes correct, no crash on full-batch trigger
[PASS] gradients flow correctly: zffn.grad populated, mixing_head.grad norm=1012.0374
```

All FOK Gate tests passed.

```
$ python smoke_test_belief_gate.py
Using device: mps
z_gated shape: (2, 8, 64), device: mps:0
weff shape: (2, 8), range: [0.8053, 0.8053]
flagged for reconsolidation: 1 entries
```

Smoke test passed.

Two of these deserve emphasis as evidence rather than mere passes. The `[INFO]` line quantifies Fix 2: under the specification's unbounded formula the weight reaches **-4.25** after 50 rounds of inhibition, while the floored version holds at **0.0200**. And the last belief-store test reproduces Fix 3's bug numerically — the old loop reports the second query as **0.6800** versus **0.5309** for the first, inventing freshness that does not exist.

3.2 Retrieval throughput benchmark

Batched retrieval speedup over the per-token loop

Ratio of wall-clock time, old loop ÷ `query_batch()` . Higher is better; the gain grows with workload.

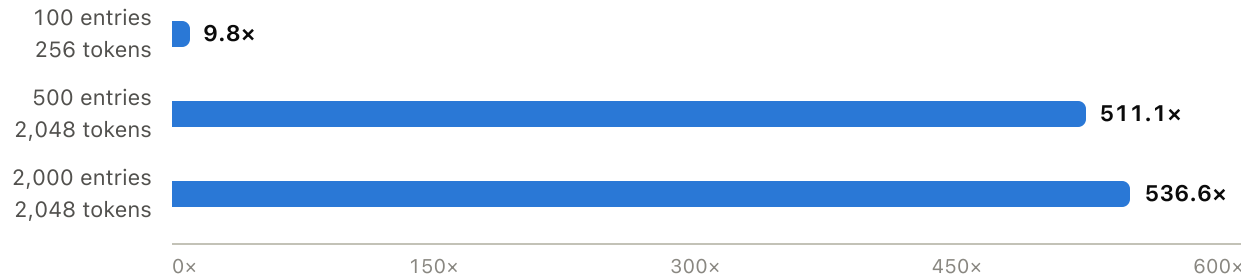


Figure 1. Speedup is modest at toy scale but exceeds 500x at realistic sequence length, because the per-token loop's interpreter overhead grows with both token count and store size while the batched matmul does not.

Table 1. Raw benchmark output — `python bench_query.py`

ENTRIES	TOKENS	PER-TOKEN LOOP	QUERY_BATCH()	SPEEDUP
100	256	0.0700 s	0.0072 s	9.8x
500	2,048	2.7947 s	0.0055 s	511.1x
2,000	2,048	11.3335 s	0.0211 s	536.6x

The practical significance: at 2,000 entries and sequence length 2,048, belief retrieval went from **11.3 seconds** per forward pass — which would make the architecture unusable — to **21 milliseconds**.

3.3 End-to-end loop composition

Beyond per-module tests, I verified that the three stages actually chain together with real tensors on the GPU: Belief Gate → FOK Gate → reconsolidation, using two seeded beliefs and a contradicting claim.

```
BeliefGate -> z_gated (2, 8, 64) on mps:0, weff range [0.3108, 0.7645], flagged=2
FOKGate    -> z_out (2, 8, 64), PIK range [0.3108, 0.7645], triggered 5/16 tokens (31%)
Reconsolid. -> {'contested': [['e24ce51c', 'be05a301']], 'reinforced': ['41938550'],
                'neutral': [], 'skipped_no_claim': [], 'skipped_no_text': []}
e1 state=CONTESTED weff=0.4050 e2 state=ACTIVE weight=0.6500

End-to-end composition OK
```

This single trace exercises every claim the architecture makes:

- **Graded confidence.** `w_eff` spans [0.3108, 0.7645] — the gate produces a real distribution of belief strengths, not a binary hit/miss.
- **Selective reasoning.** The FOK gate triggered on **5 of 16 tokens (31%)**. The reasoning module ran on under a third of the batch, which is the architecture's core efficiency claim demonstrated rather than asserted.
- **Contest, not erase.** Entry `e24ce51c` was contradicted, marked `CONTESTED`, and given successor `be05a301` — yet still retrieves at `w_eff = 0.4050`. Under the original specification it would have been 0.0 and the conflict would have been unreachable. This is Fix 1 visible in an integrated run.

- **Reinforcement.** Entry 41938550 was entailed by its claim and strengthened from 0.6 to 0.65, remaining ACTIVE.

REPOSITORY STATUS — NO GITHUB LINK YET

A local Git repository is initialised (`.git` present, branch `main`), but it currently has **zero commits and no configured remote**, so there is no GitHub URL to cite in this report. All source is on the development machine at `~/Documents/EpstLoop`. Publishing the repository is the first item in Next Steps; the module inventory in §2 and the verbatim run output above stand as the evidence of progress in the meantime.

4 Challenges

An underspecified specification

Two steps in Section 3.5 turned out not to be implementable as written. `extract_claims(output, concept_key)` presumes you can attribute a natural-language claim back to a specific belief entry from hidden states — that is a substantial NLP research problem, not a function call. Separately, the spec calls for `NLI(claim, entry.content_kv)`, but `content_kv` is an opaque sparse key-value tensor and textual NLI cannot run against it.

How I handled it: rather than fake either step, I narrowed the module's contract and documented the boundary. Claim extraction is now an *input* (`claims: dict[entry_id, str]`) supplied by the caller's generation loop, keeping this module's scope to "given a claim, run the epistemic update correctly" — the part actually specified. For the NLI mismatch I added a `content_text` field to `BeliefEntry` and run inference against that; entries without it are reported in `skipped_no_text` rather than silently mishandled.

A sign-flip bug that tests would not have caught

The unbounded RIF formula (Fix 2) fails silently. Weights drift negative only after many inhibition rounds, and the damage shows up downstream as inverted attention output — nowhere near the line that caused it. I found it by reasoning about the formula's range rather than by observing a failure.

How I handled it: I wrote a test that applies 50 rounds of inhibition and asserts the floor holds, plus a companion test that runs the *unfixed* arithmetic and asserts it goes negative. That second test is unusual but valuable — it fails if the bug ever stops being a bug, which keeps the fix's rationale from rotting.

Retrieval was fast enough to pass tests and far too slow to use

Per-token retrieval looked fine on 16-token smoke tests. At realistic sequence length it cost 11.3 s per forward pass.

How I handled it: wrote `bench_query.py` before optimising, so the gain was measured rather than assumed, then vectorised into a single `matmul` (§3.2). Validating the batched version is also what surfaced Fix 3 — the intra-pass decay reset — since the two implementations disagreed on a case where the loop was wrong.

Device and autograd friction

Two smaller obstacles. First, `torch.device("mps") != tensor.device`, which resolves to `mps:0`, so a naive equality assertion rejected correctly-placed tensors; the gate now compares by `.type` and

resolves the concrete device by querying a real tensor. Second, the FOK gate's scatter-back initially used in-place index assignment, which risks breaking the autograd graph — replaced with out-of-place `index_copy` plus `torch.where`, and covered by a test asserting `mixing_head` actually receives gradient (measured norm 1012.04). Without that test the head could have silently been untrainable.

Scope discipline

Real Quiet-STaR rationale generation and a pretrained SAE are each large components. Building either would have stalled progress on the epistemic loop itself, which is the project's actual contribution.

How I handled it: both are pluggable placeholders with the substitution boundary documented in the module docstring — a small MLP for `thought_fn`, randomly-initialised frozen weights for the SAE encoder. Each is explicitly labelled as structural, not functional, so the control flow and the compute-saving property can be validated now and the real components dropped in without touching the surrounding class.

5 Next Steps

Planned before the next progress report, in priority order:

Table 2. Planned work

#	TASK	DETAIL AND RATIONALE
1	Publish the repository	Commit the 10 modules, add a README documenting the three specification fixes, and push to GitHub so subsequent reports can cite a permanent link and commit history. Closes the one evidence gap in §3.
2	Real NLI backend	Swap <code>MockNLI</code> for <code>HFNLIClassifier</code> with DeBERTa-v3 MNLI and verify the label map matches the model card. The reconsolidation plumbing is tested; the semantics are not yet, because the mock is word-overlap only.
3	Pretrained SAE weights	Replace the randomly-initialised encoder with a GemmaScope-style SAE trained on a target model's residual stream, so concept keys become semantically meaningful and nearest-neighbour retrieval means something.
4	pytest + CI	Convert the four hand-rolled scripts to a <code>pytest</code> suite and add a GitHub Actions run. The CPU tests (12 of 17) can gate every push; the MPS tests stay local.
5	Experiment 4.2 — belief convergence	Use the summary dict from <code>run_reconsolidation_pass</code> to track contested/reinforced counts over many passes and test whether the store converges or oscillates. The instrumentation already exists; the experiment harness does not.
6	Experiment 4.3 — FOK calibration	Measure expected calibration error of the PIK signal against actual correctness, and sweep <code>theta_fok</code> to characterise the accuracy/compute trade-off. The 31% trigger rate in §3.3 is a single arbitrary-threshold data point, not yet a curve.
7	Real <code>thought_fn</code>	Implement Quiet-STaR-style parallel rationale sampling and train the mixing head. Non-trivial because the trigger is a hard threshold and therefore non-differentiable, requiring a straight-through estimator or RL.

#	TASK	DETAIL AND RATIONALE
8	ANN index (stretch)	Above roughly 10^4 entries the dense similarity matrix becomes the bottleneck again; swap in FAISS. Isolated by design — a drop-in replacement for the similarity step only, leaving weighting, state, and flagging logic untouched.

Items 1–4 are infrastructure and carry low risk. Items 5 and 6 are the first real empirical results the project will produce, and are the priority for the next report. Item 7 is the largest remaining piece of engineering and may extend beyond the next reporting period.